

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID Principle



Disusun Oleh:

Muhammad Rafly 105224040

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

SOLID Principle merupakan kumpulan lima prinsip desain dalam pemrograman berorientasi objek yang bertujuan untuk membuat struktur kode lebih rapi, mudah dikembangkan, dan tidak mudah rusak ketika terjadi perubahan. Kelima prinsip tersebut adalah Single Responsibility Principle (SRP), Open/Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP).

Pada kasus sistem KRS, permasalahan utama terjadi karena seluruh proses aplikasi bertumpu pada satu kelas besar bernama SistemKRSManger. Kelas tersebut menangani validasi prasyarat, perhitungan UKT, pencetakan PDF, hingga penyimpanan data ke database. Akibatnya, perubahan kecil pada satu bagian dapat merusak bagian lain yang tidak berhubungan.

Program yang dibuat pada tugas ini merupakan rancangan ulang dari sistem KRS tersebut. Sistem dipecah menjadi beberapa class dengan tanggung jawab yang lebih jelas. Validasi KRS dipisahkan ke KRSValidator, pencetakan draft KRS dipisahkan ke KRSPDFGenerator, penyimpanan data dipisahkan melalui KRSRepository, dan perhitungan UKT dipisahkan menggunakan interface UKTCalculator. Dengan rancangan ini, sistem menjadi lebih fleksibel untuk menerima perubahan seperti migrasi dari MySQL ke NoSQL serta penambahan skema UKT baru seperti MBKM.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	daftarMataKuliah	List<MataKuliah>	Menyimpan daftar mata kuliah yang dipilih.
2	validator	KRSValidator	Menangani validasi prasyarat dan batas SKS.
3	repository	KRSRepository	Menangani penyimpanan riwayat KRS.
4	pdfGenerator	KRSPDFGenerator	Menangani pencetakan draf KRS.
5	uktCalculator	UKTCalculator	Menangani strategi perhitungan UKT.
6	databaseConnection	DatabaseConnection	Menjadi abstraksi penyimpanan MySQL atau NoSQL.

III. Constructor dan Method

NO	Nama Metode	Jenis Metode	Fungsi
1	Mahasiswa()	Constructor	Menginisialisasi data mahasiswa.
2	MataKuliahTeori()	Constructor	Menginisialisasi mata kuliah teori.
3	MataKuliahPraktikum()	Constructor	Menginisialisasi mata kuliah praktikum.
4	MataKuliahKKN()	Constructor	Menginisialisasi mata kuliah KKN.
5	KRS()	Constructor	Membuat objek KRS berdasarkan mahasiswa.
6	KRSRepository()	Constructor	Menghubungkan repository dengan database.
7	SistemKRSManger()	Constructor	Menginisialisasi validator, repository, PDF generator, dan kalkulator UKT.
8	tambahMataKuliah()	Procedural	Menambahkan mata kuliah ke KRS.
9	hitungTotalSks()	Functional	Menghitung total SKS.
10	buatRingkasan()	Functional	Membuat ringkasan data KRS.
11	validasiPrasyarat()	Functional	Memvalidasi data mahasiswa dan mata kuliah.
12	validasiBatasSks()	Functional	Mengecek batas maksimal SKS.

13	hitungUKT()	Functional	Menghitung UKT sesuai jalur masuk.
14	simpanDataKRS()	Procedural	Menyimpan data KRS ke database.
15	cetakDrafPDF()	Procedural	Mencetak draf KRS.
16	prosesPengisianKRS()	Procedural	Menjalankan alur utama pengisian KRS.

IV. Dokumentasi dan Pembahasan Code

a. Isi Kode

Mahasiswa.java

```

1  public class Mahasiswa {
2      private String nim;
3      private String nama;
4      private String jalurMasuk;
5
6      public Mahasiswa(String nim, String nama, String jalurMasuk) {
7          this.nim = nim;
8          this.nama = nama;
9          this.jalurMasuk = jalurMasuk;
10     }
11
12     public String getNim() {
13         return nim;
14     }
15
16     public String getNama() {
17         return nama;
18     }
19
20     public String getJalurMasuk() {
21         return jalurMasuk;
22     }
23 }

```

MataKuliah.java

```

1  public interface MataKuliah {
2      String getNama();
3      int getSks();
4      void tampilkanInfo();
5  }

```

OperasiPraktikum.java

```

public interface OperasiPraktikum {
    void alokasiAsistenLab();
    void cekPeralatanPraktikum();
}

```

MataKuliahTeori.java

```

public class MataKuliahTeori implements MataKuliah {
    private String nama;
    private int sks;

    public MataKuliahTeori(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }

    public String getNama() {
        return nama;
    }

    public int getSks() {
        return sks;
    }

    public void tampilkanInfo() {
        System.out.println("Mata kuliah teori: " + nama + " (" + sks + " SKS)");
    }
}

```

MataKuliahPraktikum.java

```

public class MataKuliahPraktikum implements MataKuliah, OperasiPraktikum {
    private String nama;
    private int sks;
    public MataKuliahPraktikum(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }
    public String getNama() {
        return nama;
    }
    public int getSks() {
        return sks;
    }
    public void tampilkanInfo() {
        System.out.println("Mata kuliah praktikum: " + nama + " (" + sks + " SKS)")
    }
    public void alokasiAsistenLab() {
        System.out.println("Asisten lab dialokasikan untuk " + nama);
    }
    public void cekPeralatanPraktikum() {
        System.out.println("Peralatan praktikum dicek untuk " + nama);
    }
}

```

MataKuliahKKN.java

```

public class MataKuliahKKN implements MataKuliah {
    private String nama;
    private int sks;

    public MataKuliahKKN(String nama, int sks) {
        this.nama = nama;
        this.sks = sks;
    }

    public String getNama() {
        return nama;
    }

    public int getSks() {
        return sks;
    }

    public void tampilkanInfo() {
        System.out.println("Mata kuliah lapangan/KKN: " + nama + " (" + sks + " SKS)");
    }

    public void cekLokasiKegiatan() {
        System.out.println("Lokasi kegiatan KKN dicek untuk " + nama);
    }
}

```

KRS.java

```
import java.util.ArrayList;
import java.util.List;
public class KRS {
    private Mahasiswa mahasiswa;
    private List<MataKuliah> daftarMataKuliah = new ArrayList<>();
    public KRS(Mahasiswa mahasiswa) {
        this.mahasiswa = mahasiswa;
    }
    public void tambahMataKuliah(MataKuliah mataKuliah) {
        daftarMataKuliah.add(mataKuliah);
    }
    public Mahasiswa getMahasiswa() {
        return mahasiswa;
    }
    public List<MataKuliah> getDaftarMataKuliah() {
        return daftarMataKuliah;
    }
    public int hitungTotalSks() {
        int total = 0;
        for (MataKuliah mk : daftarMataKuliah) {
            total += mk.getSks();
        }
        return total;
    }
    public void tampilkanRingkasan() {
        System.out.println("NIM          : " + mahasiswa.getNim());
        System.out.println("Nama          : " + mahasiswa.getNama());
        System.out.println("Jalur Masuk : " + mahasiswa.getJalurMasuk());
        System.out.println("Daftar Mata Kuliah:");
        for (MataKuliah mk : daftarMataKuliah) {
            System.out.println("- " + mk.getNama() + " (" + mk.getSks() + " SKS)");
        }
        System.out.println("Total SKS   : " + hitungTotalSks());
    }
}
```

KRSValidator.java

```
public class KRSValidator {
    private final int BATAS_MAKSIMAL_SKS = 24;
    public boolean validasiPrasyarat(Mahasiswa mahasiswa, MataKuliah mataKuliah) {
        return mahasiswa != null && mataKuliah != null;
    }
    public boolean validasiBatasSks(KRS krs, MataKuliah mataKuliah) {
        return krs.hitungTotalSks() + mataKuliah.getSks() <= BATAS_MAKSIMAL_SKS;
    }
}
```

UKTCalculator.java

```
public interface UKTCalculator {
    double hitungUKT(KRS krs);
}
```

UKTRegulerCalculator.java

```
public class UKTRegulerCalculator implements UKTCalculator {
    public double hitungUKT(KRS krs) {
        return krs.hitungTotalSks() * 250000;
    }
}
```

UKTKaryawanCalculator.java

```
public class UKTKaryawanCalculator implements UKTCalculator {
    public double hitungUKT(KRS krs) {
        return krs.hitungTotalSks() * 300000 + 500000;
    }
}
```

UKTInternasionalCalculator.java

```
public class UKTInternasionalCalculator implements UKTCalculator {
    public double hitungUKT(KRS krs) {
        return krs.hitungTotalSks() * 500000 + 2000000;
    }
}
```

UKTBidikmisiCalculator.java

```
public class UKTBidikmisiCalculator implements UKTCalculator {
    public double hitungUKT(KRS krs) {
        return 0;
    }
}
```

UKTMBKMCALculator.java

```
public class UKTMBKMCALculator implements UKTCalculator {
    public double hitungUKT(KRS krs) {
        return krs.hitungTotalSks() * 150000;
    }
}
```

DatabaseConnection.java

```
public interface DatabaseConnection {
    void simpanDataKRS();
}
```

MySQLDatabaseConnection.java

```
public class MySQLDatabaseConnection implements DatabaseConnection {
    public void simpanDataKRS() {
        System.out.println(x: "Data KRS disimpan ke database MySQL.");
    }
}
```

NoSQLDatabaseConnection.java

```
public class NoSQLDatabaseConnection implements DatabaseConnection {
    public void simpanDataKRS() {
        System.out.println(x: "Data KRS disimpan ke Cloud NoSQL.");
    }
}
```

KRSRepository.java

```
public class KRSRepository {
    private DatabaseConnection databaseConnection;
    public KRSRepository(DatabaseConnection databaseConnection) {
        this.databaseConnection = databaseConnection;
    }
    public void simpanRiwayat(KRS krs) {
        System.out.println("Menyimpan riwayat KRS mahasiswa " + krs.getMahasiswa().getNama());
        databaseConnection.simpanDataKRS();
    }
}
```

KRSPDFGenerator.java

```

public class KRSPDFGenerator {
    public void cetakDrafPDF(KRS krs, double totalUKT) {
        System.out.println(x: "==== DRAF KRS PDF =====");
        krs.tampilkanRingkasan();
        System.out.println("Total UKT      : Rp" + totalUKT);
        System.out.println(x: "=====");
    }
}

```

SistemKRSManger.java

```

import java.util.List;

public class SistemKRSManger {
    private KRSValidator validator;
    private KRSRepository repository;
    private KRSPDFGenerator pdfGenerator;
    private UKTCalculator uktCalculator;
    public SistemKRSManger(KRSValidator validator, KRSRepository repository, KRSPDFGenerator pdfGenerator, UKTCalculator uktCalculator) {
        this.validator = validator;
        this.repository = repository;
        this.pdfGenerator = pdfGenerator;
        this.uktCalculator = uktCalculator;
    }
    public void prosesPengisianKRS(Mahasiswa mahasiswa, List<MataKuliah> daftarPilihan) {
        KRS krs = new KRS(mahasiswa);
        System.out.println(x: "=== Proses Pengisian KRS Dimulai ===");
        for (MataKuliah mk : daftarPilihan) {
            if (!validator.validasiPrasyarat(mahasiswa, mk)) {
                System.out.println("Prasyarat tidak terpenuhi untuk " + mk.getNama());
                continue;
            }
            if (!validator.validasiBatasSks(krs, mk)) {
                System.out.println("Mata kuliah " + mk.getNama() + " ditolak karena melebihi batas SKS.");
                continue;
            }
            krs.tambahMataKuliah(mk);
        }
        double totalUKT = uktCalculator.hitungUKT(krs);
        pdfGenerator.cetakDrafPDF(krs, totalUKT);
        repository.simpanRiwayat(krs);
        System.out.println(x: "=== Proses Pengisian KRS Selesai ===");
    }
}

```

Main.java

```

import java.util.ArrayList;
import java.util.List;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Mahasiswa mahasiswa = new Mahasiswa(nim: "105224040", nama: "Rafly", jalurMasuk: "MBKM");
        MataKuliah teori = new MataKuliahTeori(nama: "Pemrograman Berorientasi Objek", sks: 3);
        MataKuliahPraktikum praktikum = new MataKuliahPraktikum(nama: "Praktikum PBO", sks: 2);
        MataKuliah knn = new MataKuliahKKN(nama: "Kuliah Kerja Nyata", sks: 2);
        praktikum.alokasiAsistenLab();
        praktikum.cekPeralatanPraktikum();
        List<MataKuliah> daftarPilihan = new ArrayList<>();
        daftarPilihan.add(teori);
        daftarPilihan.add(praktikum);
        daftarPilihan.add(knn);
        DatabaseConnection database = new NoSQLDatabaseConnection();
        KRSRepository repository = new KRSRepository(database);
        KRSValidator validator = new KRSValidator();
        KRSPDFGenerator pdfGenerator = new KRSPDFGenerator();
        UKTCalculator uktCalculator = new UKTMBKMCalculator();
        SistemKRSManger manager = new SistemKRSManger(validator, repository, pdfGenerator, uktCalculator);
        manager.prosesPengisianKRS(mahasiswa, daftarPilihan);
    }
}

```


b. Alur Program

1. Program dimulai dari class Main, kemudian objek Mahasiswa dibuat terlebih dahulu dengan data NIM, nama, dan jalur masuk. Pada contoh ini, jalur masuk yang digunakan adalah MBKM karena sistem dirancang agar dapat menerima skema UKT baru.
2. Setelah data mahasiswa dibuat, program membuat beberapa objek mata kuliah, yaitu MataKuliahTeori, MataKuliahPraktikum, dan MataKuliahKKN. Ketiga objek ini digunakan untuk menunjukkan bahwa sistem dapat menangani beberapa jenis mata kuliah yang berbeda.
3. Untuk mata kuliah praktikum, program menjalankan method alokasiAsistenLab() dan cekPeralatanPraktikum(). Method ini hanya dimiliki oleh MataKuliahPraktikum, sehingga mata kuliah teori dan KKN tidak dipaksa menjalankan operasi praktikum.
4. Selanjutnya, semua mata kuliah yang dipilih dimasukkan ke dalam List<MataKuliah>. Daftar ini nantinya akan diproses oleh SistemKRSManger sebagai daftar pilihan KRS mahasiswa.
5. Program kemudian menentukan jenis database yang digunakan melalui interface DatabaseConnection. Pada contoh ini, objek yang digunakan adalah NoSQLDatabaseConnection, sehingga sistem menunjukkan bahwa penyimpanan data dapat diarahkan ke NoSQL tanpa mengubah alur utama program.
6. Setelah itu, program membuat objek pendukung seperti KRSRepository, KRSValidator, KRSPDFGenerator, dan UKTMBKMCalculator. Objek-objek ini diberikan ke dalam SistemKRSManger melalui constructor.
7. Ketika method prosesPengisianKRS() dijalankan, sistem membuat objek KRS baru berdasarkan data mahasiswa. Kemudian setiap mata kuliah pada daftar pilihan akan divalidasi menggunakan KRSValidator.
8. Jika mata kuliah lolos validasi prasyarat dan total SKS tidak melebihi batas maksimal, maka mata kuliah tersebut akan ditambahkan ke dalam KRS. Jika tidak lolos validasi, mata kuliah akan ditolak dan program melanjutkan pengecekan mata kuliah berikutnya.
9. Setelah proses validasi selesai, sistem menghitung total UKT menggunakan UKTCalculator. Karena mahasiswa menggunakan jalur MBKM, maka perhitungan dilakukan oleh class UKTMBKMCalculator.
10. Tahap terakhir, sistem mencetak draf KRS melalui KRSPDFGenerator, lalu menyimpan riwayat KRS melalui KRSRepository. Repository kemudian meneruskan proses penyimpanan ke database yang sedang digunakan, yaitu NoSQLDatabaseConnection.
11. Dengan alur ini, SistemKRSManger tidak lagi mengerjakan seluruh proses sendirian. Class tersebut hanya mengatur jalannya program, sedangkan validasi, perhitungan UKT, pencetakan PDF, dan penyimpanan data sudah dipisahkan ke class masing-masing.

c. Penerapan Prinsip SOLID dalam Alur Program

Penerapan prinsip SOLID pada program ini terlihat dari pemisahan tugas pada setiap class. Prinsip Single Responsibility Principle (SRP) diterapkan karena SistemKRSManger tidak lagi menangani semua proses sendiri. Proses validasi dipisahkan ke KRSValidator, pencetakan draf KRS dipisahkan ke KRSPDFGenerator, dan penyimpanan data dipisahkan ke KRSRepository.

Prinsip Open/Closed Principle (OCP) diterapkan pada bagian perhitungan UKT. Program menggunakan interface UKTCalculator, sehingga setiap jalur masuk memiliki class perhitungan masing-masing. Dengan cara ini, penambahan skema baru seperti MBKM dapat dilakukan tanpa mengubah class utama.

Prinsip Liskov Substitution Principle (LSP) dan Interface Segregation Principle (ISP) diterapkan pada bagian mata kuliah. MataKuliahTeori, MataKuliahPraktikum, dan MataKuliahKKN tetap dapat digunakan sebagai objek MataKuliah. Operasi khusus praktikum dipisahkan ke interface OperasiPraktikum, sehingga mata kuliah teori dan KKN tidak dipaksa memiliki method praktikum.

Prinsip Dependency Inversion Principle (DIP) diterapkan pada bagian database. KRSRepository tidak bergantung langsung pada MySQLDatabaseConnection, melainkan pada interface DatabaseConnection. Hal ini membuat sistem dapat menggunakan MySQL atau NoSQL tanpa mengubah alur utama program.

V. Kesimpulan

Berdasarkan program yang dibuat, penerapan prinsip SOLID membuat sistem KRS menjadi lebih rapi dan mudah dikembangkan. Setiap class memiliki tugas yang lebih jelas, sehingga perubahan pada satu bagian tidak langsung mengganggu bagian lainnya.

Program juga menjadi lebih fleksibel karena perhitungan UKT dibuat dalam class yang berbeda untuk setiap jalur masuk. Selain itu, pemisahan interface pada mata kuliah membuat class teori, praktikum, dan KKN hanya memiliki method yang sesuai dengan kebutuhannya. Penggunaan interface DatabaseConnection juga membuat sistem lebih siap untuk migrasi dari MySQL ke NoSQL.